

DEEPPFINERY / OPEN SOURCE / APACHE 2.0



OpenHarness

Runs an army of agents.

Learns from every try.

Keeps every action safe.

A self-hosted agent studio and execution harness for models, tools, knowledge and machines.

v0.2.0

Open Harness API

MCP-native

Self-hosted

Your agents, on your infrastructure.

Design agent workflows on a canvas, connect them to models, tools, knowledge and machines, and run them from a chat, an API, a webhook or a schedule.

ONE TOOL PROTOCOL

Everything arrives through MCP.

Paste a server URL, sign in or add a key, pick the tools each agent may call. A new integration is a new MCP server, not new code.

DURABLE BY DESIGN

Runs outlive their runner.

Queued, checkpointed and resumed on another runner, without blindly replaying a step that acted on the world.

ONE STACK

One Docker Compose file.

API, runner, studio, MongoDB, RabbitMQ, Weaviate and device gateway. No agent framework between you and the model.

One stack for a fleet of agents.

01

Every pattern

ReAct, plan and execute, reflection and autonomous loops, with effort budgets

02

Army of agents

Chains, parallel groups of 8, routing, and up to 12 sub-agents per run

03

Learns every try

Feedback and failures become lessons the next run recalls

04

Shared memory

Knowledge bases, a shared workspace and per-run task memory

05

Machines

Linux, containers, Windows and Chrome, sandboxed, no public IP

06

Layered safety

Deny by default, schema checks, signed hooks, full audit trail

07

Durable runs

Checkpoints, saga-style resume and idempotency keys on every call

08

Open standards

MCP for every tool and the Open Harness API, 35 of 35 conformance tests

Four patterns, all of them bounded.

ReAct

Default. Each turn: call tools or answer. Results feed the next turn.

Plan and execute

Write a numbered plan, run each step with tools, then synthesize.

Reflection

Draft, critique as a strict reviewer, revise. One to three rounds.

Autonomous loop

Iterate, carrying progress forward, until the done marker or the limit.

Effort	Turns/pass	Tokens	Time
Light	4	30k	90 s
Medium	12	120k	5 min
High	24	400k	10 min
Extra high	36	1M	15 min
Max	40	5M	15 min

Auto picks light, medium or high per request. A spent budget ends with one final call without tools, never a runaway loop.

An army of agents, on one budget.

8

agents working in parallel in one workflow step

12

sub-agents per run, spawned 4 at a time

7

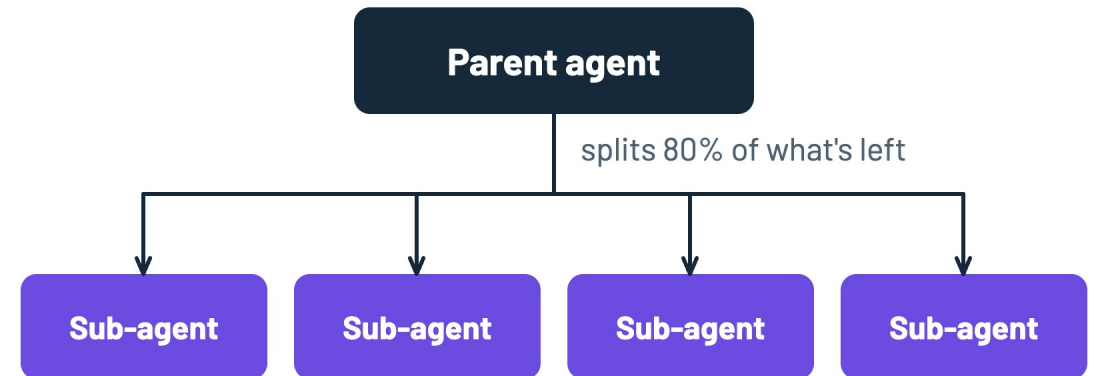
step types, from agents to conditions and email

1

token budget that bounds the whole tree

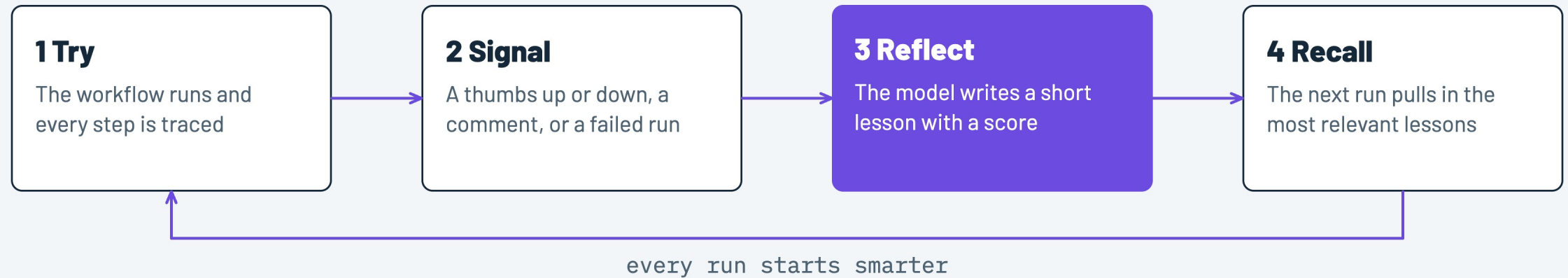
Build the team on a canvas

- Chain agents step by step
- Fan out to a parallel group, then merge
- Route work to a specialist with a condition
- Loop with a bounded step budget, up to 500
- Start from 8 starter workflows



Each sub-agent gets a fresh context, one skill and a subset of the parent's tools

Agents learn from every try.



In context, not in weights

The model never changes. Lessons are notes with provenance that people can review, edit or delete.

Ready for RL and fine-tuning

Every rated run and lesson exports as JSONL with input, output, feedback and score.

Memory that outlives a single run.

LONG-TERM

Knowledge bases

- Markdown notes plus PDF, DOCX, CSV and more
- Hybrid search with cited passages
- Weaviate, Qdrant, OpenSearch, Elasticsearch or any Vector Stores API

SHARED

Knowledge workspace

- Agents search, read and write with kb_search, kb_read and kb_write
- Every write is a new note, so parallel agents never collide
- Decisions keep their reasons

PER RUN

Task memory

- A scratch notebook for analysis and handoffs
- Large tool results are saved there, the agent keeps a preview
- Gone when the run ends

Agents with hands on real machines.



Linux hosts

Commands, files,
search, processes

Containers

Sandboxes with a hardened
Docker install

Windows

Adds UI Automation

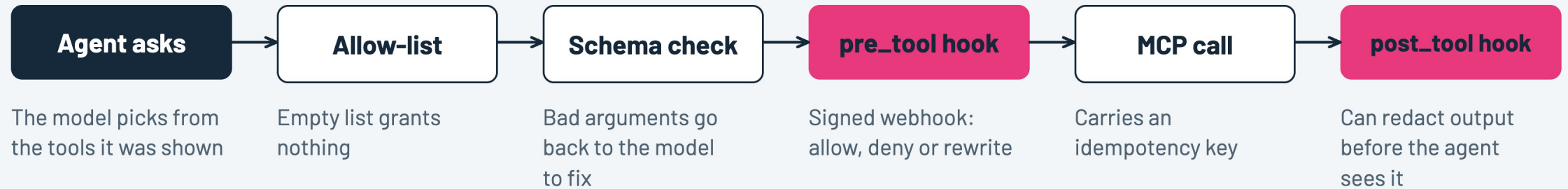
Chrome

Tabs, clicks, typing, network

Enforced on the machine itself

- Commands run as argv, never through a shell
- rm, sudo and shutdown denied by default
- File paths locked to a work directory
- Read-only mode, output caps and timeouts
- A local audit log on every machine

Every tool call passes one gate.



Fails closed

If a tool hook times out or errors, the call is blocked. The agent is told why and the tool never runs.

Least privilege

Nothing the model says grants access. Tools, knowledge and tenant are set by configuration before it runs.

Traced as it happens

Every call, argument, result and hook decision is written to the run trace.

Seven layers of safety.

Layer	What it stops
Tenancy and keys	Per-tenant isolation, workflow-scoped API keys, rate-limited login
Prompt hygiene	Retrieved text is labeled as reference data, never as instructions
Tool gate	Deny-by-default allow-lists and schema checks on every call
Signed hooks	Allow, deny or rewrite any tool call; blocked if the hook fails
Machine policy	No shell, dangerous commands denied, paths sandboxed, read-only mode
Network and secrets	Private egress blocked, DNS-rebinding checks, encrypted credentials
Durability and audit	No replay of side effects, hard budgets, every step traced

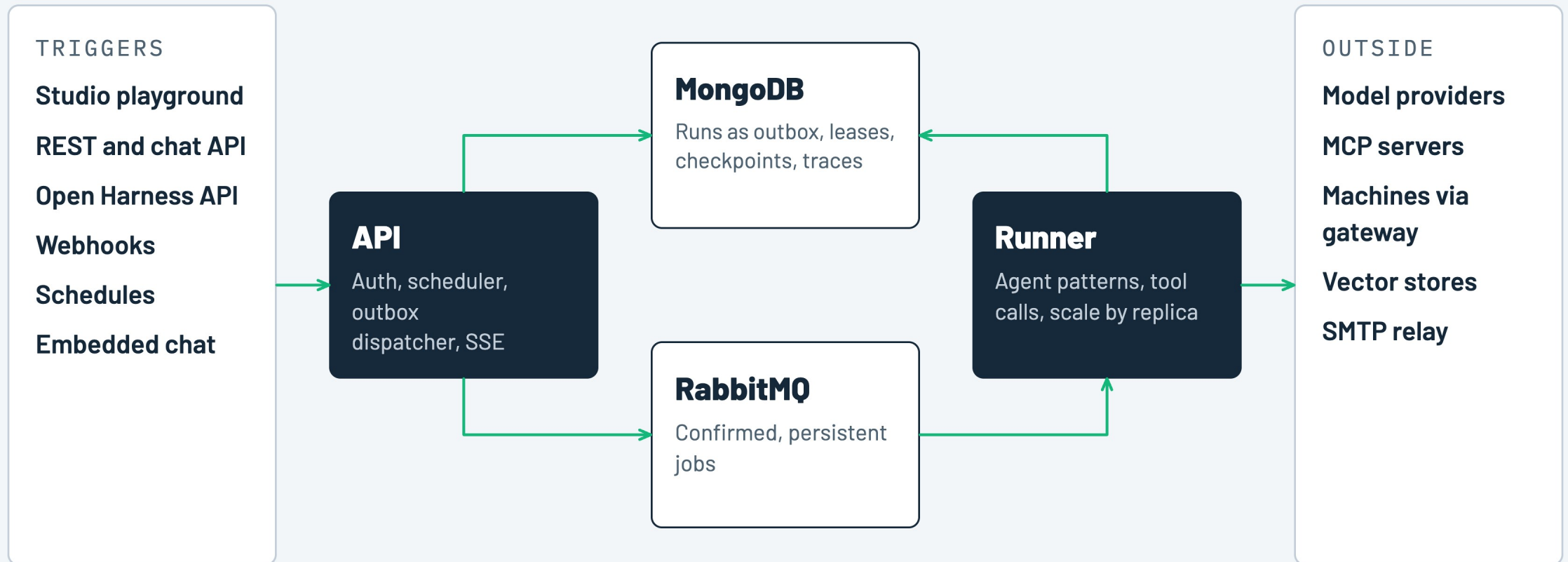
NEXT

Human approval before risky actions

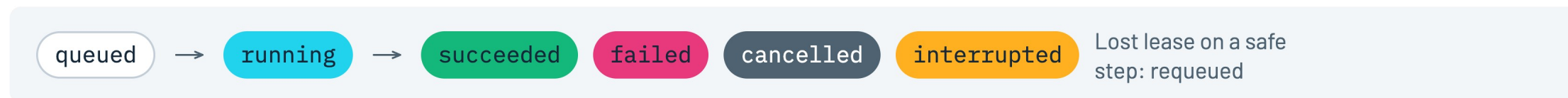
NVIDIA NeMo Guardrails

Exploring: NVIDIA OpenShell

Every run takes one durable path.



A crash never replays a side effect.



Idempotent submission

Same Idempotency-Key and payload returns the original run; a changed payload gets 409.

Outbox before broker

The run is saved as queued before publish. If RabbitMQ is down, the dispatcher republishes.

Exclusive lease

Atomic claim with a lease ID, a 5 s heartbeat and a 30 s expiry. Duplicate deliveries no-op.

Checkpointed resume

A cursor before every step, output after. A new runner continues at the cursor.

Side-effect-aware policy

Default "safe": resume only steps with no external effect; otherwise stop as interrupted, with the reason.

A key on every tool call

`_meta.idempotencyKey = runId:nodeId:call` lets MCP servers make a replay harmless.

The Open Harness API, implemented.

35/35

tests passing in the spec's own
conformance suite

115

spec routes, all mounted from a table
generated from the spec

0

silent gaps: an unbuilt route answers 501,
never a guess

How it stays honest

- Pinned to spec 0.2.0 at upstream commit f727de1
- Capability manifest built from the registry that mounts routes
- Unknown route: 404. Unbuilt route: 501 with its name
- Extensions live under x-openharness, never in spec fields

SERVED TODAY

Executions and streaming

Agents (OAF)

Tools

Hooks and events

Webhooks

Capability manifest

Models

COMING NEXT

Remaining routes for skills, MCP servers, sessions, memory, sub-agents, files and planning (issues #7 to #25)

Spec concepts map onto a real engine.

Open Harness spec	In OpenHarness
Harness	One installation, served at /openharness/v1/harnesses/openharness
Agent	A workflow; its entry agent carries model, system prompt and tools_access
Execution	A durable run. queued → pending, succeeded → completed, interrupted → failed
Stream	text, tool_call_start/end, tool_result, progress, error, then done exactly once with usage
Reconnect	Last-Event-ID resumes without repeats; streams replay for one hour
Agent package	Open Agent Format zip: AGENTS.md, skills, MCP configs. Credentials never export
Hooks	pre_tool and post_tool may allow, deny or modify; HMAC-signed, fail closed
Errors	Spec envelope with code, message, domain and operation

Where it stands, and what comes next.

v0.2.0

Apache 2.0, one Docker Compose stack, one-command upgrade

35/35

Open Harness conformance tests passing

4

test layers: unit, real-stack fault injection, Chromium and WebKit, conformance

NEXT ON THE ROADMAP

Human approval before risky actions

NVIDIA NeMo Guardrails safety

pgvector as a native vector store driver

Remaining Open Harness routes (#7 to #25)